FEAT-ZD-48600 · DES-01 Platform Architecture Update [1] · Platform architecture change (when flagged)

Design agent · item feat.platform_update · source hobasa-design-agent/outputs/FEAT-ZD-48600/checks/DES-01_feat.platform_update.md · approved by automated checks and independent judge

feat.platform_update: FEAT-ZD-48600 (DES-01 check record)

DES-01 ran in **check mode** for this item. The spine reported "condition not met: TRD flags a platform-level change" (record:intake). This record checks that against the inputs. It sets out what was checked, the evidence, the conclusion, and what would make the action needed. Nothing was changed in the platform architecture document, code, configuration, tests or environments.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief; feat.trd
Step	DES-01 Platform Architecture Update (check mode, attempt 1)	record:brief
Workbook action	If the feature needs an architectural change at the platform level, update the platform architecture document first	record:brief
Inputs read	feat.trd (approved at REQ-07); org.architecture (feature override)	record:brief
Other files read	`org/org.architecture.md` (org-wide baseline); `org/org.decision_log.md`	file:hobasa-agentic-sdlc/org/org.architecture.md; file:hobasa-agentic-sdlc/org/org.decision_log.md
Result	**No platform architecture update needed now.** The platform document is left unchanged.	Sections 2 and 3 below

1. What a platform-level change means for this check

A change is platform-level when it changes something org.architecture lays down for every repository, not just one repository's code. For this item, org.architecture lays down six things:

#	Platform element in org.architecture	Platform statement
P-1	Topology	Four repositories in one flow: ap_pipeline → ap_forecast → ap_aging → ap_portal
P-2	Runtimes	ap_pipeline, ap_forecast, ap_aging: Python 3.9; ap_portal: Node.js 22 CommonJS
P-3	Responsibility per repository	ap_pipeline produces Invoice records; ap_forecast predicts payment delay; ap_aging computes aging; ap_portal renders HTML from JSON
P-4	Inter-service contract rule	"JSON between services carries amounts as strings with two decimals and dates as ISO strings"
P-5	Compatibility rule	"Keep every existing signature; add new functions."
P-6	Named public functions per repository	ingest.parse_feed, transform.to_model, dq.quality_report, pipeline.run; predict_delay_days, build_explanation_request; aging_buckets, days_past_due, bucket_for; renderAgingSummary

Source: org.architecture. The org-wide baseline adds that ap_aging has "No database, no network, no secrets" and that "existing function signatures and return shapes must not change" (file:hobasa-agentic-sdlc/org/org.architecture.md).

2. Checks and evidence

Each row compares what the approved TRD does with the platform element it could affect.

#	Check	Evidence in feat.trd	Platform element	Finding
K-1	New repository, service or removal of one?	Repositories stay ap_pipeline, ap_forecast, ap_aging and ap_portal. Components C-1 to C-10 all sit in these four, plus a cross-repository test harness (C-10).	P-1	No change
K-2	New runtime, language version or framework?	F-15 keeps Python 3.9 and Node.js 22 CommonJS, and tells the lanes to use no syntax newer than 3.9. Tests use the existing `node:test` and repository CI (F-14).	P-2	No change; no new technology
K-3	Repository responsibilities shifted?	Delay histories come from ap_pipeline (C-2), the risk rule from ap_forecast (C-4), the overdue list from ap_aging (C-6, C-7), and rendering from ap_portal (C-8). Each fits that repository's stated role.	P-3	No change
K-4	Inter-service contract rule changed?	CT-1 (IF-6) sends `amount` as a two-decimal string. It has no date field, and any date added later "must be an ISO string". `days` is an integer and `risk` is a string.	P-4	Conforms. CT-1 is a feature contract under the existing rule, not a new rule
K-5	Any existing signature or return shape changed?	Only two existing functions change, and both additively: `to_model` gets a new record key and reject reason with the same signature and return shape (IF-1), and `build_explanation_request` gets `risk=None` as its last parameter (IF-4). Everything else is a new function. The TRD's validation check records "Pass" against the signature rule (section 9).	P-5	Conforms

#	Check	Evidence in feat.trd	Platform element	Finding
K-6	New persistent storage, database, network path or secret?	"No storage, schema migration or new persistent data. Every new function is in-memory and pure" (section 3). S-6: no secrets or credentials added. ap_aging's new functions are pure (IF-5.8, C-7).	Baseline "No database, no network, no secrets"	No change
K-7	Tenancy, authentication, finance posting or payment execution?	"Not affected: finance posting, authentication, tenancy, payment execution, LLM provider or model, aging export" (section 1). S-1 repeats this. Tier T2, advisory.	Platform security and tenancy boundary	No change
K-8	External provider or model changed?	Provider and model still come from `models.json` unchanged (IF-4.4). `vendor_risk` makes no LLM call (IF-3.2, S-3). The only new outbound data is the risk label in the prompt (S-5, TQ-8).	Platform external dependencies	No provider or model change. See note N-2
K-9	Decision-log conflict that needs a platform rule?	New code follows D-2026-031, D-2026-044 and D-2026-050 (TRD section 9). The two conflicts found are in existing code: `bucket_for` (F-10, TQ-1) and `formatAmount` (F-12, TQ-2). Each is local to one repository and is raised as its own question.	file:hobasa-agentic-sdlc/org/org.decision_log.md	No platform rule needed
K-10	Does the TRD itself flag a platform-level change?	No section of the TRD asks for a platform architecture change. Its own validation (section 9) records the signature rule as Pass and finds no conflict with an approved output.	—	Not flagged. This matches the spine's condition result (record:intake)

3. Conclusion

The DES-01 action is not needed for FEAT-ZD-48600 now. The feature keeps all six platform elements (P-1 to P-6): the topology, runtimes, repository roles, the JSON contract rule, the compatibility rule and the security boundary. Every change is a new function or an additive extension, which org.architecture explicitly allows ("add new functions"). The feature-level contract CT-1 and the new `paid_date` record field belong in the design document (DES-02), not in the platform architecture document.

Notes for DES-02 and the Architect (non-blocking; none of them changes the conclusion):

#	Note	Source
N-1	P-6 lists each repository's public functions. After delivery, that list will not name the new functions: `delays_by_vendor`, `vendor_risk`, `overdue_with_risk`, `overdue_rows_to_json` and `renderOverdueRisk`. org.architecture allows new functions, so this is a catalogue refresh, not an architectural change. It is raised as open question 1.	org.architecture; feat.trd (C-2, C-4, C-6, C-7, C-8)
N-2	S-5 / TQ-8: the explanation prompt would also carry the vendor's risk label to the configured provider. No provider or model changes. If the Architect treats this as a change to the platform's external data flow, it becomes a trigger (T-4 below).	feat.trd (S-5, TQ-8)
N-3	org.architecture says ap_pipeline "produces Invoice records". The TRD found there is no `Invoice` class: records are plain dicts (F-2). This wording does not conflict with the feature, so no update is made.	org.architecture; feat.trd (F-2)
N-4	The org-wide baseline describes ap_aging invoices as dicts with id, due_date, amount and paid, with no `vendor`. The new `overdue_with_risk` reads `vendor` (IF-5.7). This is the input of a new function, so the existing contract is unchanged.	file:hobasa-agentic-sdlc/org/org.architecture.md; feat.trd (F-11, IF-5.7)

4. What would trigger the action

DES-01 should run in full (and update the platform architecture document) if any of these happens before or during design and build:

#	Trigger	Linked open item
T-1	The answer to TQ-3 introduces a new service, API or transport layer between ap_aging and ap_portal, or moves serialisation ownership across repositories in a way that changes the "JSON between services" rule	feat.trd (TQ-3, R-7, IF-6)
T-2	The fix for `bucket_for` (TQ-1) or `formatAmount` (TQ-2) is made a platform-wide rule, or changes an existing signature or return shape	feat.trd (TQ-1, TQ-2)
T-3	The feature adds persistent storage, a database, a network call from the libraries, a secret, or anything touching tenancy or authentication	feat.trd (section 3, S-1, S-6)
T-4	The Architect classes sending the risk label to the external provider (TQ-8) as a change to the platform's external data flow, or the provider or model changes (for example through PROV-ACME-2026-11, PQ-5)	feat.trd (S-5, TQ-8, R-3)
T-5	A new runtime, language version or repository is introduced, or an existing signature is changed rather than extended	org.architecture; feat.trd (F-15, IF-1, IF-4)
T-6	The cross-repository test harness (IF-8) becomes a deployed component, not a test asset	feat.trd (IF-8, C-10, R-9)

5. Open questions (non-blocking)

1. Should the platform document's function catalogue (P-6) be refreshed after release to name the new public functions? This is a documentation refresh only (N-1).
2. Architect: does the answer to TQ-3 or TQ-8 change the platform's inter-service or external data flow? If so, DES-01 is re-run (T-1, T-4).